

WeatherAnalyser – Wetterdaten-Auswertung

CSV-Import · MergeSort · Min/Max/Durchschnitt · 100 % Unit-Test-Abdeckung

Programmiersprache: Python 3 | Lehrbuch: Kapitel 1–4 (Algorithmen)

PROJEKTÜBERSICHT

Projektname	WeatherAnalyser – Wetterdaten-Auswertung
Auftraggeber	Bergbahnen Flumserberg AG, Flums
Lehrjahr / Klasse	1. Lehrjahr – Applikationsentwicklung
Technologie	Python 3 (kein Framework, kein Jupyter Notebook)
Sozialform	Solo

ECKDATEN

Dauer	4 Wochen à 4 Lektionen (= 16 Lektionen total)
Startdatum	KW ____ / ____ (Lehrperson einzutragen)
Abgabedatum	KW ____ / ____ (Lehrperson einzutragen)
Version	1.0
Status	Freigegeben
Erstellt am	19. April 2026

MODULBEZUG

Modul 319	Applikationen entwerfen und implementieren – HZ 1, 2, 3, 4, 5, 6
Modul 122	Abläufe mit einer Scriptsprache automatisieren – HZ 1, 2, 3, 4, 5

LEHRBUCH-BEZUG (Algorithmen, Kap. 1–4)

Kapitel 1	Binäre Suche, Landau-Notation, Python-Grundlagen
Kapitel 2	Arrays vs. Listen, Selectionsort – Grundlage für MergeSort
Kapitel 3	Rekursion: Basisfall + Rekursionsfall – MergeSort ist rekursiv
Kapitel 4	Teile-und-herrsche-Verfahren – das Prinzip hinter MergeSort

BEWERTÜNGSÜBERSICHT (IPT-2.1)

Produkt (50 %)	F.01 – F.04 (individuell bewertet)
Dokumentation (30 %)	F.05, F.06
Lernjournal (20 %)	Ü.01 – Ü.04

ABGABEN

- ✓ Lauffähiges Python-Programm inkl. Beispiel-CSV (GitHub oder ZIP)
- ✓ Wöchentliches Lernjournal (4 × Journaleintrag)
- ✓ Videopräsentation (2–5 Min.: Demo + Erklärung + Reflexion)
- ✓ Projektdokumentation mit User Stories (alle 5 Phasen)

Bezug zum Lehrbuch «Algorithmen» – Kapitel 1–4

Dieses Projekt baut direkt auf den vier Kapiteln des Lehrbuchs auf. Die folgende Tabelle zeigt, welche Konzepte aus dem Buch du im Projekt konkret anwendest. Halte beim Programmieren das Buch griffbereit und notiere im Lernjournal, welche Buchstellen dir besonders geholfen haben.

Kapitel	Inhalt aus dem Buch	Anwendung im Projekt
Kap. 1	Binäre Suche, Landau-Notation ($O \log n$ vs $O n^2$), Python-Grundlagen	Laufzeit von MergeSort ($O n \log n$) erklären und im Vergleich einordnen
Kap. 2	Arrays (Listen) vs. verkettete Listen, Selectionsort $O(n^2)$	Python-Listen als Datenstruktur; Vergleich MergeSort $O(n \log n)$ vs. Selectionsort $O(n^2)$
Kap. 3	Rekursion: Basisfall + Rekursionsfall, Call Stack	<code>merge_sort()</code> ist eine rekursive Funktion – Basisfall und Rekursionsfall explizit benennen
Kap. 4	Teile-und-herrsche-Verfahren: teilen → lösen → zusammenführen	MergeSort = Teile-und-herrsche: Liste halbieren → sortieren → <code>merge()</code>

1. Kundenbriefing



Unternehmen: Bergbahnen Flumserberg AG, Flums

Die Bergbahnen Flumserberg AG betreibt im Kanton St. Gallen ein ausgedehntes Ski- und Wandergebiet mit mehreren Bergstationen auf bis zu 2 220 m ü. M. An jeder Station sind automatische Wettermessstationen installiert, die stündlich Temperatur, Windgeschwindigkeit und Schneehöhe aufzeichnen. Diese Daten werden als CSV-Dateien lokal gespeichert.

Die Betriebsleiterin Frau Sandra Walser schildert das Problem: «Wir haben Tausende von Messwerten, aber kein einfaches Werkzeug, um schnell zu sehen, was die tiefste Temperatur der letzten Woche war, wie stark der Wind im Durchschnitt geblasen hat oder an welchem Tag die meiste Schneehöhe gemessen wurde.» Das Technik-Team öffnet die CSV-Dateien bisher manuell in Excel und sucht per Hand – was bei mehreren Hundert Zeilen viel zu lange dauert.

Frau Walser wünscht sich: Ein Python-Skript, das eine CSV-Datei einliest, die Messwerte mit MergeSort sortiert und automatisch Minimum, Maximum und Durchschnitt berechnet und anzeigt. Alle Funktionen müssen mit automatisierten Tests abgesichert sein.

Auftrag an dich

Du arbeitest als Applikationsentwicklerin / Applikationsentwickler bei den Bergbahnen Flumserberg AG und hast den Auftrag erhalten, das Python-Skript «WeatherAnalyser» zu entwickeln. Das Programm liest eine CSV-Datei mit Wettermesswerten ein, sortiert die Daten mit MergeSort und berechnet für jede Messgrösse Minimum, Maximum und Durchschnitt. Alle Funktionen werden mit Unit-Tests auf 100 % Anweisungsüberdeckung geprüft.

2. Auftrag & Lernziele

Auftrag (aus Kundensicht)

Entwickle ein Python 3-Script mit folgenden Funktionen:

- `csv_einlesen(dateipfad)` – liest eine CSV-Datei ein und gibt eine Liste von Dictionaries zurück (Datum, Temperatur, Wind, Schneehöhe). Fehlerbehandlung wenn Datei fehlt oder leer ist.
- `merge_sort(liste, schluessel)` – sortiert die Liste aufsteigend nach einem wählbaren Schlüssel (z. B. 'temperatur') mit dem MergeSort-Algorithmus (rekursiv, Teile-und-herrsche).
- `minimum(liste, schluessel)` – gibt den Datensatz mit dem kleinsten Wert zurück.
- `maximum(liste, schluessel)` – gibt den Datensatz mit dem grössten Wert zurück.
- `durchschnitt(liste, schluessel)` – berechnet den Mittelwert, gerundet auf 2 Dezimalstellen. Gibt None zurück bei leerer Liste.
- `auswertung(dateipfad)` – Hauptfunktion: ruft alle obigen Funktionen auf und gibt einen formatierten Bericht im Terminal aus.
- Vollständige Unit-Tests (pytest) mit 100 % Anweisungsüberdeckung für alle Funktionen ausser `auswertung()`.

Schulische Lernziele

Lernziel	Kompetenz
Du kannst eine CSV-Datei mit Python einlesen, parsen und als strukturierte Liste von Dictionaries bereitstellen.	F.03 / M122-HZ2,3
Du kannst MergeSort nach dem Teile-und-herrsche-Prinzip (Kapitel 4) rekursiv in Python implementieren.	F.01 / M319-HZ4
Du kannst den Basisfall und den Rekursionsfall von MergeSort (Kapitel 3) korrekt definieren und erklären.	F.02 / M319-HZ1
Du kannst Statistikfunktionen als eigenständige, testbare Unterfunktionen umsetzen.	F.03 / M319-HZ4
Du kannst Unit-Tests mit pytest schreiben und 100 % Anweisungsüberdeckung (pytest-cov) erreichen.	F.04 / M319-HZ6
Du kannst Python-Code mit Docstrings kommentieren und ein Script als CLI-Tool ausführen.	F.05 / M122-HZ3,5
Du kannst dein Projekt in 5 Phasen planen und vollständig dokumentieren.	F.06 / M431-HZ3

Kompetenzabdeckung

Kompetenz	Wie wird sie abgedeckt?
F.01	<code>merge_sort()</code> mit Teile-und-herrsche nach Kap. 4, automatisiert getestet
F.02	Rekursion (Kap. 3): Basisfall + Rekursionsfall von <code>merge_sort()</code> erklären; Vergleich mit Selectionsort (Kap. 2)
F.03	EVA: Eingabe CSV, Verarbeitung Sort+Statistik, Ausgabe Terminal – alle Python-Kontrollstrukturen
F.04	100 % Statement Coverage mit <code>pytest-cov</code> , Testprotokoll mit min. 16 Testfällen
F.05	Docstrings (Args, Returns, Raises) über allen Funktionen; Script via <code>python weather_analyser.py</code>
F.06	User Stories mit AK, PAP-Diagramme, Planungstabelle, alle 5 Phasen dokumentiert

3. Rahmenbedingungen

Punkt	Details
Dauer	4 Wochen à 4 Lektionen = 16 Lektionen total
Sozialform	Solo
Programmiersprache	Python 3.10 oder neuer
Testframework	pytest + pytest-cov (Installation: <code>pip install pytest pytest-cov</code>)
Dateiformat	CSV mit Semikolon als Trennzeichen (Vorlage in Abschnitt 4)
Abgabe	Code + Beispiel-CSV (GitHub/ZIP), Lernjournal (4 Einträge), Video (2–5 Min.), Doku
Projektphasen	Analyse → Design → Planung → Umsetzung → Test
Erlaubte Hilfsmittel	Unterlagen, Lehrbuch (Kap. 1–4), Internet, AI-Tools mit Quellenangabe
Abgabedatum	wird von der Lehrperson eingetragen

Python-Umgebung einrichten

Terminal-Befehle zum Start
Virtuelle Umgebung erstellen (empfohlen)
<code>python3 -m venv .venv</code>
<code>source .venv/bin/activate</code> # macOS/Linux
<code>.venv\Scripts\activate</code> # Windows
Abhängigkeiten installieren
<code>pip install pytest pytest-cov</code>
Programm starten
<code>python weather_analyser.py</code>
Tests mit Coverage ausführen
<code>pytest test_weather_analyser.py --cov=weather_analyser --cov-report=term-missing</code>

CSV-Dateiformat (verbindlich)

Semikolon-getrennt. Erste Zeile = Kopfzeile (wird beim Einlesen übersprungen).

Beispiel: wetterdaten.csv			
datum;	temperatur;	windgeschwindigkeit;	schneehoehe
2024-01-15;	-8.2;	34;	120
2024-01-16;	-3.1;	18;	118
2024-01-17;	-12.5;	52;	125
2024-01-18;	-1.8;	11;	119
2024-01-19;	-6.4;	29;	122
2024-01-20;	-15.3;	61;	130
2024-01-21;	-4.7;	22;	128

4. Detaillierte Aufgabenstellung

Die 4 Wochen sind in 5 Projektphasen unterteilt. Halte die Reihenfolge ein und dokumentiere jede Phase, bevor du mit der nächsten anfängst.

Teilaufgabe 1 – Analyse / Grobkonzept: Problemstellung & User Stories	
Beschreibung	Lies das Kundenbriefing sorgfältig und lies parallel dazu Kapitel 1 (Einführung in Algorithmen) im Lehrbuch. Beantworte schriftlich: Was genau will Frau Walser? Welche Daten enthält die CSV? Was sind Ein- und Ausgaben jeder Funktion? Welche Fehlerfälle gibt es (Datei nicht vorhanden, leere Datei, fehlerhafter Wert)? Erkläre in eigenen Worten, was Landau-Notation bedeutet, und beschreibe die Laufzeit von MergeSort ($O(n \log n)$) im Vergleich zu Selectionsort ($O(n^2)$).
Erwartetes Ergebnis	Analysedokument (mind. 1–2 Seiten), 4 User Stories, Funktionsübersicht mit Ein-/Ausgaben, Landau-Notation von MergeSort erklärt
Kompetenz	F.06 / M319-HZ1 / M431-HZ1
Tipp / Buchbezug	Lehrbuch Kap. 1 (S. 19–38): Laufzeitvergleich einfache Suche vs. binäre Suche – dasselbe Prinzip gilt für Selectionsort vs. MergeSort.

Teilaufgabe 2 – Design / Detailkonzept: Ablaufdiagramme & Datenstruktur	
Beschreibung	Lies Kapitel 3 (Rekursion) und Kapitel 4 (Quicksort / Teile-und-herrsche) im Lehrbuch, bevor du mit dem Design anfängst. Zeichne je einen Programmablaufplan für <code>merge_sort()</code> (inklusive <code>merge()</code>) und für <code>auswertung()</code> . Zeige klar den Basisfall und den rekursiven Aufruf. Definiere alle 6 Funktionen in einer Tabelle: Name, Parameter, Rückgabety, Fehlerfälle. Beschreibe das Dictionary-Format, das <code>csv_einlesen()</code> zurückgibt. Erkläre in 4–6 Sätzen, wie MergeSort das Teile-und-herrsche-Prinzip umsetzt («divide → conquer → combine» nach Kap. 4).
Erwartetes Ergebnis	2 PAP-Diagramme (<code>merge_sort</code> + <code>auswertung</code>), Funktionstabelle, Dictionary-Beispiel, Teile-und-herrsche-Erklärung mit Buchbezug
Kompetenz	F.01 / F.02 / M319-HZ2 / M122-HZ1
Tipp / Buchbezug	Lehrbuch Kap. 3 (S. 57–69): Basisfall-Definition. Kap. 4 (S. 71–88): Teile-und-herrsche-Landstück-Beispiel → direkte Analogie zum MergeSort-Halbieren.

Teilaufgabe 3 – Planung: Aufgaben, Meilensteine & Wochenplan	
Beschreibung	Erstelle eine detaillierte Aufgabenliste für alle 4 Wochen (16 Lektionen). Schätze jede Aufgabe in Lektionen. Definiere 3 Meilensteine: M1 « <code>csv_einlesen()</code> funktioniert, Daten korrekt geparkt» (Ende Woche 1), M2 « <code>merge_sort()</code> und Statistikfunktionen laufen korrekt» (Ende Woche 3), M3 «100 % Test-Coverage, Dokumentation fertig, Video abgegeben» (Ende Woche 4).
Erwartetes Ergebnis	Planungstabelle mit Aufgaben, Lektionenschätzung und 3 Meilensteinen
Kompetenz	F.06 / M431-HZ2,3
Tipp / Buchbezug	Plane Woche 4 fast vollständig für Tests, Dokumentation und Video. <code>merge()</code> separat zu implementieren kostet mehr Zeit als erwartet.

Teilaufgabe 4 – Umsetzung: Python-Implementierung

Beschreibung	Implementiere alle 6 Funktionen in einer Datei <code>weather_analyser.py</code> . Anforderungen: (1) Jede Funktion hat einen Docstring ("""...""") mit Args, Returns und Raises. (2) <code>csv_einlesen()</code> liest mit <code>open()</code> und <code>csv.reader()</code> oder manuell ein, überspringt die Kopfzeile und gibt eine Liste von Dictionaries zurück. (3) <code>merge_sort()</code> teilt die Liste rekursiv in zwei Hälften und führt sie mit einer separaten <code>merge()</code> -Hilfsfunktion zusammen (Basisfall: Liste mit 0 oder 1 Element). (4) <code>minimum()</code> und <code>maximum()</code> arbeiten ohne Python-Builtins <code>min()/max()</code> – iteriere selbst durch die Liste. (5) <code>durchschnitt()</code> gibt <code>None</code> zurück bei leerer Liste (kein <code>ZeroDivisionError</code>). (6) <code>auswertung()</code> gibt einen formatierten Bericht aus: z. B. «Temperatur: Min -15.3 °C (2024-01-20) Max -1.8 °C Ø -7.43 °C». (7) <code>if __name__ == '__main__':</code> am Ende startet <code>auswertung()</code> .
Erwartetes Ergebnis	Lauffähige <code>weather_analyser.py</code> , korrekte Ausgabe im Terminal, Docstrings über allen Funktionen, Beispiel-CSV im Projektordner
Kompetenz	F.01 / F.02 / F.03 / F.05 / M319-HZ3,4,5 / M122-HZ2,3
Tipp / Buchbezug	Implementiere zuerst <code>csv_einlesen()</code> und teste manuell mit <code>print()</code> . Dann <code>merge_sort()</code> – erst <code>split()</code> , dann <code>merge()</code> separat. Lehrbuch Kap. 4 (S. 80–88): Quicksort-Code als Analogie für die Struktur.

Teilaufgabe 5 – Test: Unit-Tests mit pytest und 100 % Anweisungsüberdeckung

Beschreibung	Schreibe eine Testdatei <code>test_weather_analyser.py</code> . Verwende eine kleine Test-CSV (<code>test_wetterdaten.csv</code>) für <code>csv_einlesen()</code> -Tests. Teste mindestens: <code>csv_einlesen()</code> Normalfall + Fehlerfall (Datei fehlt), <code>merge_sort()</code> mit gemischten, sortierten, umgekehrt sortierten, 1-Element- und leeren Listen, <code>minimum()</code> , <code>maximum()</code> , <code>durchschnitt()</code> mit bekannten Testdaten, <code>durchschnitt()</code> mit leerer Liste (→ <code>None</code>). Führe <code>pytest --cov=weather_analyser --cov-report=term-missing</code> aus und prüfe 100 % Statements. Fülle das Testprotokoll vollständig aus.
Erwartetes Ergebnis	<code>test_weather_analyser.py</code> mit min. 16 Testfällen, <code>test_wetterdaten.csv</code> , Coverage-Report (100 % Statements), ausgefülltes Testprotokoll
Kompetenz	F.04 / M319-HZ6 / M122-HZ4
Tipp / Buchbezug	<code>pytest</code> erwartet Funktionsnamen mit <code>test_</code> -Präfix. Fehlerfall für <code>csv_einlesen()</code> : <code>with pytest.raises(FileNotFoundError): csv_einlesen('x.csv')</code> . Lehrbuch Kap. 1: «Teste deinen Code – immer.»

5. User Stories (F.06)

Mindestens 4 User Stories mit Akzeptanzkriterien sind Pflicht.

User Story 1

Als Betriebsleiterin (Frau Walser) **möchte ich** die Temperaturdaten einer Woche automatisch auswerten lassen können **damit** ich sofort sehe, wann es am kältesten und wärmsten war, ohne Excel zu öffnen.

Akzeptanzkriterien:

- ☒ Das Script liest eine CSV-Datei ein und gibt Min/Max/Ø der Temperatur aus.
- ☒ Die Ausgabe ist klar lesbar und enthält Datum, Wert und Einheit (z. B. «Min: -15.3 °C am 2024-01-20»).
- ☒ Bei fehlender oder leerer CSV-Datei erscheint eine verständliche Fehlermeldung – kein Absturz.

User Story 2

Als Pistenwart **möchte ich** die Schneehöhen-Daten sortiert nach Wert sehen können **damit** ich schnell erkenne, an welchem Tag die meiste Schneehöhe gemessen wurde.

Akzeptanzkriterien:

- ☒ `merge_sort()` sortiert die Liste korrekt nach einem wählbaren Schlüssel.
- ☒ Das Ergebnis ist aufsteigend sortiert – der Tag mit der geringsten Schneehöhe steht zuerst.
- ☒ `merge_sort()` verändert die ursprüngliche Liste nicht (Python-Kopie).

User Story 3

Als Stationsleiter **möchte ich** den Durchschnitt der Windgeschwindigkeit der letzten Woche kennen **damit** ich Trends erkennen und Sicherheitsmassnahmen rechtzeitig planen kann.

Akzeptanzkriterien:

- ☒ `durchschnitt()` berechnet den Mittelwert aller Windgeschwindigkeits-Werte korrekt.
- ☒ Das Ergebnis ist auf 2 Dezimalstellen gerundet.
- ☒ Bei einer leeren Liste gibt die Funktion None zurück – kein ZeroDivisionError.

User Story 4

Als zukünftige Entwicklerin / zukünftiger Entwickler **möchte ich** den Code ohne Rückfragen verstehen und weitere Messgrößen ergänzen können **damit** das Script einfach wartbar und erweiterbar bleibt.

Akzeptanzkriterien:

- ☒ Jede Funktion hat einen Docstring mit Args, Returns und Raises.
- ☒ Das Design-Dokument erklärt MergeSort als Teile-und-herrsche mit PAP-Diagramm.
- ☐ Alle Funktionen sind mit pytest-Tests abgedeckt (100 % Statement Coverage).

User Story 5 – Eigene User Story (von dir zu ergänzen)

Als _____ möchte ich _____ damit _____.

Akzeptanzkriterien:

☐ _____

☐ _____

☐ _____

6. Wöchentliches Lernjournal

Fülle das Lernjournal am Ende jeder Projektwoche aus (10–15 Minuten). Notiere bei Frage 4 immer, welche Kapitel im Lehrbuch dir diese Woche geholfen haben.

Woche 1 – Datum: <u>20.4.2026</u>	
Fokus: Analyse, Design, Planung + csv_einlesen() implementieren (Lehrbuch Kap. 1+2)	
1. Was habe ich diese Woche konkret gemacht? <i>2–3 Sätze – nicht «ich habe programmiert», sondern was genau.</i>	Repository in Debian erstellt und mit Github verbunden
2. Was hat gut funktioniert? <i>Konkret: z. B. «Das Einlesen der CSV-Datei hat beim ersten Versuch funktioniert.»</i>	Ich hatte keine Probleme alles vorzubereiten
3. Was war schwierig oder unklar? <i>Ehrlich! Z. B. «Der merge()-Schritt von MergeSort war schwieriger als erwartet.»</i>	
4. Was habe ich dadurch gelernt? <i>Was weißt du jetzt, was du vorher nicht wusstest? Bezug zum Lehrbuch (Kapitel)?</i>	Ich weiss wie man Repositories erstellt und die verbindet und kann Git commands benutzen
5. Wie schätze ich meine eigene Arbeit ein? (Ü.04) <i>Note 1–6 mit kurzer Begründung</i>	5 Ich habe zu viel Zeit verbraucht mit dem Vorbereiten mit Github
6. Was nehme ich mir für nächste Woche vor? <i>1–2 konkrete Ziele</i>	Teilaufgaben anfangen

Woche 2 – Datum: <u>27.4.2026</u>	
Fokus: merge_sort() und merge() – Rekursion verstehen (Lehrbuch Kap. 3+4)	
1. Was habe ich diese Woche konkret gemacht? <i>2–3 Sätze – nicht «ich habe programmiert», sondern was genau.</i>	An Teilaufgabe 1 und 2 gearbeitet Mehr über Merge Sort und Merge herausgefunden
2. Was hat gut funktioniert? <i>Konkret: z. B. «Das Einlesen der CSV-Datei hat beim ersten Versuch funktioniert.»</i>	Merge Sort in ein PAP Diagramm umzuwandeln hat gut geklappt
3. Was war schwierig oder unklar? <i>Ehrlich! Z. B. «Der merge()-Schritt von MergeSort war schwieriger als erwartet.»</i>	
4. Was habe ich dadurch gelernt? <i>Was weißt du jetzt, was du vorher nicht wusstest? Bezug zum Lehrbuch (Kapitel)?</i>	Ich weiss jetzt wie Merge sort und Merge besser funktioniert
5. Wie schätze ich meine eigene Arbeit ein? (Ü.04) <i>Note 1–6 mit kurzer Begründung</i>	4 Ich habe nicht alle Teilaufgaben geschafft
6. Was nehme ich mir für nächste Woche vor? <i>1–2 konkrete Ziele</i>	Mit dem Python Programm beginnen und Teilaufgabe 3 fertig machen

Woche 3 – Datum: 4.5.2026

Fokus: Statistikfunktionen, auswertung(), erste automatisierte Tests

1. Was habe ich diese Woche konkret gemacht?

2–3 Sätze – nicht «ich habe programmiert», sondern was genau.

Python Skript geschrieben und Kommentare geschrieben wie der Code funktioniert

2. Was hat gut funktioniert?

Konkret: z. B. «Das Einlesen der CSV-Datei hat beim ersten Versuch funktioniert.»

Der Code hat gut funktioniert und die Grundkriterien erfüllt

3. Was war schwierig oder unklar?

Ehrlich! Z. B. «Der merge()-Schritt von MergeSort war schwieriger als erwartet.»

Ein paar Zeilen vom Code waren nicht genau verständlich

4. Was habe ich dadurch gelernt?

Was weißt du jetzt, was du vorher nicht wusstest? Bezug zum Lehrbuch (Kapitel)?

Ich weiss jetzt wie man Dateien einlesen tut mit Python

5. Wie schätze ich meine eigene Arbeit ein? (Ü.04)

Note 1–6 mit kurzer Begründung

4 Ich habe viel KI benutzt um den Code zu schreiben und möchte beim nächsten Projekt weniger KI benutzen

6. Was nehme ich mir für nächste Woche vor?

1–2 konkrete Ziele

Tests durchführen und das Video erstellen

Woche 4 – Datum: 11.5.2026

Fokus: Tests vervollständigen, 100 % Coverage, Testprotokoll, Dokumentation, Video

1. Was habe ich diese Woche konkret gemacht?

2–3 Sätze – nicht «ich habe programmiert», sondern was genau.

Test vervollständigt und Coverage ausgeführt

2. Was hat gut funktioniert?

Konkret: z. B. «Das Einlesen der CSV-Datei hat beim ersten Versuch funktioniert.»

Viele Tests haben ohne Probleme funktioniert

3. Was war schwierig oder unklar?

Ehrlich! Z. B. «Der merge()-Schritt von MergeSort war schwieriger als erwartet.»

Ein paar Tests haben nicht funktioniert und ich musste ein paar Sachen im Code ändern

4. Was habe ich dadurch gelernt?

Was weißt du jetzt, was du vorher nicht wusstest? Bezug zum Lehrbuch (Kapitel)?

Ich habe gelernt wie Tests mit "pytest" funktionieren

5. Wie schätze ich meine eigene Arbeit ein? (Ü.04)

Note 1–6 mit kurzer Begründung

4 Der Coverage Test tut nicht 100 % erreichen sonst lief alles gut

6. Was nehme ich mir für nächste Woche vor?

1–2 konkrete Ziele

Das Video fertig machen und die restlichen Dateien in GitHub hochladen

7. Abschlussreflexion & Videopräsentation



Videopräsentation (2–5 Minuten)

Nimm deinen Bildschirm auf. Deine Präsentation besteht aus drei Teilen:

Inhalt	
Teil 1 – Demo (ca. 1 Min.)	Starte das Script im Terminal: <code>python weather_analyser.py</code> . Zeige die Ausgabe mit einer echten CSV-Datei. Demonstriere auch einen Fehlerfall (z. B. Datei nicht vorhanden).
Teil 2 – Technische Erklärung (ca. 2 Min.)	Zeige deinen Code. Erkläre EINE Stelle im Detail – zum Beispiel: Wie funktioniert <code>merge()</code> Schritt für Schritt? Was ist der Basisfall von <code>merge_sort()</code> ? Oder: Wie hängt MergeSort mit dem Teile-und-herrsche-Verfahren aus Kapitel 4 zusammen?
Teil 3 – Reflexion (ca. 1–2 Min.)	Was war schwieriger als erwartet? Was hast du aus dem Lehrbuch (Kap. 1–4) gelernt, das dir im Projekt konkret geholfen hat? Was würdest du beim nächsten Projekt anders machen? (Ü.01, Ü.04)

8. Testprotokoll-Vorlage (F.04)

Fülle dieses Protokoll nach Abschluss der Testphase aus und füge den Coverage-Report bei.

Test-ID	Funktion	Eingabe / Szenario	Erwartet	Resultat	Status
T01	csv_einlesen	wetterdaten.csv (7 Zeilen)	Liste mit 7 Dicts		☒ OK ☐ Fail
T02	csv_einlesen	Datei nicht vorhanden	FileNotFoundError		☒ OK ☐ Fail
T03	csv_einlesen	Nur Kopfzeile (leer)	Leere Liste []		☒ OK ☐ Fail
T04	merge_sort	[3,1,4,1,5] nach Wert	[1,1,3,4,5]		☒ OK ☐ Fail
T05	merge_sort	Bereits sortiert [1,2,3]	[1,2,3]		☒ OK ☐ Fail
T06	merge_sort	Umgekehrt [5,4,3,2,1]	[1,2,3,4,5]		☒ OK ☐ Fail
T07	merge_sort	1-Element-Liste [42]	[42]		☒ OK ☐ Fail
T08	merge_sort	Leere Liste []	[]		☒ OK ☐ Fail
T09	merge_sort	Dict-Liste nach 'temp'	korrekt sortiert		☒ OK ☐ Fail
T10	minimum	Testdaten-Liste	Dict mit kleinstem Wert		☒ OK ☐ Fail
T11	minimum	1-Element-Liste	Dieses Element		☒ OK ☐ Fail
T12	maximum	Testdaten-Liste	Dict mit grösstem Wert		☒ OK ☐ Fail
T13	maximum	Alle Werte gleich	Diesen Wert		☒ OK ☐ Fail
T14	durchschnitt	[10, 20, 30]	20.00		☒ OK ☐ Fail
T15	durchschnitt	Leere Liste []	None		☒ OK ☐ Fail
T16	durchschnitt	[3.333, 6.667]	5.00 (gerundet)		☒ OK ☐ Fail
T17	Eigener Test				☐ OK ☐ Fail
T18	Eigener Test				☐ OK ☐ Fail

Coverage-Nachweis (Screenshot oder Textausgabe pytest-cov einfügen):

```
pytest --cov=weather_analyser --cov-report=term-missing
```

Statements: _____% Branches: _____% Functions: _____% Lines: _____%